QFwork.ai: Payment Implementation Plan

Database: Neon Postgres. Hosting: Render. Payment gateway: not yet chosen (placeholder). October 2026.

1. Approach

- No user accounts. A payment mints a **paid access code** holding a credit balance, redeemed through the existing access gate.
- Flow: pricing card → `POST /api/checkout` → provider checkout page → webhook mints code → `checkout-return.html` polls status, shows the code and auto-redeems it.
- Every external dependency sits behind a placeholder, so the whole flow can be built and tested before any sign-up.

2. Placeholders

Placeholder	Stands in for	To go live
PGlite: Postgres in-process, local file in `.data/`	Neon Postgres	Set `DATABASE_URL`
Mock provider: fake checkout page with Pay / Cancel	Stripe / Airwallex / KPay	Add `providers/<name>.js`, set `PAYMENT_PROVIDER` + keys
`PLANS` env var with dummy HKD prices	Real pricing	Edit the env var

Safeguard: in production the server will not start without `DATABASE_URL`, and refuses the mock provider unless `ALLOW_MOCK_PAYMENTS=true`.

3. Build steps (one commit each)

1. Database layer: no behaviour change

- **db.js** (new): `pg` pool if `DATABASE_URL` is set, else PGlite; creates tables; one-time import of `trial-codes-used.json`.
- **access.js**: JSON state replaced by queries; `consumeRun` becomes one atomic UPDATE (no double-spend); add `refundRun` (fixes failed Tavus calls not refunding one-time codes).
- **server.js**: `await` at call sites; `db.init()` before startup; 503 if the database is down (fail closed).

2. Paid tier

- **access.js**: `paid` tier, balance on the code, full report, `TAVUS_MAX_CALL_SECONDS_PAID`, high-entropy codes (QF-7KQ2-M9XD-4TPA).

3. Payments backend (mock provider)

- **payments.js** + **providers/mock.js**: `createCheckout`, `verifyWebhook`, `getStatus`.
- **server.js**: `POST /api/payments/webhook` (raw body, before `express.json()`, idempotent), `POST /api/checkout`, `GET /api/checkout/status`.

4. UI

- Pricing section + nav link (exam.html); "Buy access" in gate and trial-spent screen; "Get full reports" on locked cards.
- New `checkout-return.html` and `mock-checkout.html`; remaining balance in top bar; paid code remembered in localStorage.

5. Admin + testing

- Admin endpoints behind `ADMIN_TOKEN`: list codes, check balance, issue code manually, export.
- Test: duplicate / late webhook, Tavus-failure refund, zero balance, restart, two devices on one code.

6. Later

- Real gateway adapter; optional email delivery of codes.

4. Your to-dos

When	Action
Now	Nothing. Steps 1-5 run locally on placeholders.
Before deploying step 1	Sign up at neon.tech; project in **Singapore** (match Render region). Create branches **main** (prod) and **dev** (testing). Render → Environment: `DATABASE_URL` = main string, `ADMIN_TOKEN` = long random string. Optionally put dev string in local .env.
Before real money	Choose gateway + complete HK merchant onboarding; send test keys. Register webhook URL and add secret on Render. Set real `PLANS` prices (check Tavus per-minute cost). Publish Terms / Refund / Privacy pages. Do one real HK$ purchase and refund it.
Optional	Render Starter instance so the app does not sleep (no longer needed for storage).